Trabajo Práctico Nro. 3 - 2026

Algoritmos y Estructuras de Datos



Índice

INSTRUCCIONES DEL PROGRAMA 3

1. Introducción. .....**¡Error! Marcador no definido.**
2. Estructura de Datos de los archivos. 2
3. Programación de los juegos.....3
   A. Juego del menor-mayor:.....3
   B. Número secreto:.....4
   C. Blackjack:.....4
   D. Par o impar:.....5
   E. Reporte:.....6
   F. Administración de Juegos:.....6
   G. Salir del programa: 7

1. Introducción

En este TP3 se introduce el **concepto de registros y archivos**. Deben **codificar en lenguaje Python**. Se requiere la implementación de **validaciones en cada proceso**. Ante cualquier incumplimiento de las condiciones establecidas, el **sistema deberá emitir mensajes informativos y claros al usuario** informando de lo ocurrido.

La declaración de registros y archivos deberá realizarse con el formato enseñado y aplicado en clase**. Ejemplo:**

```
class NombreDelRegistro:
        def __init__(self):
                self.campo1 = 0  #entero
                self.campo2 = " "   #string
                self.campo3 = [0]*3  #Arreglo unidimensional de enteros
                self.campo4= 0.00   #real
                self.campo5 = [[0.00] * 2 for i in range(2)]   #Matriz 2*2
                self.campo6 = True  #boolean
```

2. Modelo de datos

En este TP3 vamos a trabajar con la totalidad del modelo de datos (todos archivos físicos, almacenamiento permanente de la información):

Categorías.dat ***(****El NroCategoría es un número consecutivo secuencial desde 1 en adelante.****)***	**Opciones.dat**	**Jugadores.dat** *(sin ningún orden)*
NroCategoria: int NombreCategoria: str (30 caracteres) Pregunta: str (200 caracteres) Estado: str (A/I) #Activa Inactiva *(Una categoría tiene una única pregunta)*	NroCategoria: int NroOpción: int objeto: str (100) valor: int *(mínimo habrá 6 registros de opciones por cada categoría)*	Nombre: str(30) Creditos: float Juegos: matriz de 2 x 4 de enteros

La columna de la matriz del campo “juegos” indica el número de juego, la fila 0 la cantidad de veces que se ganó a un juego y la fila 1 la cantidad de veces que se perdió a dicho juego donde, columna 0: mayor/menor, columna 1: adivinar número, columna 2: Blackjack y columna 3: par/impar

NOTA: Al inicio de cada juego se debe pedir el nombre del jugador, se deberá registrar el jugador en el archivo JUGADORES.DAT con la cantidad de veces que ganó y que perdió y con la

acumulación de créditos que ganó o perdió. Si el nombre del jugador ya existe no se deberá crear otro jugador con el mismo nombre sino actualizar el registro. Todos los jugadores que se crean con un crédito inicial de 10.000.

Importante: cuando se entregue el trabajo, los archivos categorias.dat y opciones.dat ya deben existir y contener al menos 3 categorías y 10 opciones por categoría permitiendo que durante la defensa se puedan agregar más categorías y opciones.

3. Programación de juegos

INSTRUCCIONES DEL PROGRAMA

Desarrollo de Suite de Juegos "Azar y Lógica" en Python

Cuando el programa inicie debe mostrar un cartel, grande, dibujado con caracteres, que llame la atención del usuario donde se indique que ***los juegos de apuesta están prohibidos para los menores y es perjudicial para la salud. (****O similar)*

Luego de que el usuario o jugador hace clic en la tecla “Enter”, dicho cartel debe borrarse y en su lugar aparecer un menú de opciones iterativo como el siguiente:

Menú de Opciones

a. Juego del menor-mayor
b. Adivinar el número secreto
c. Blackjack
d. Par o impar
e. Reporte
f. Administración de Juegos
g. Salir del programa

Nota: Se deberá realizar al menos un procedimiento por cada juego.

En todos los casos, cuando un juego inicie se deberá solicitar el nombre del jugador. Si el nombre existe, no se deberá crear un nuevo registro, sino que se actualizará el registro de ese jugador según corresponda y como se detalla a continuación en cada juego.

A. Juego del menor-mayor:



Antes de iniciar la partida, se le preguntará al jugador cuantos créditos quiere apostar.



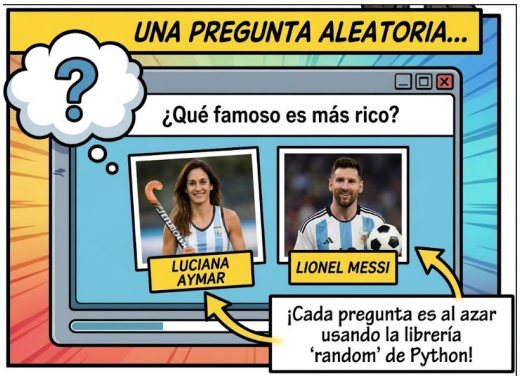
Al comenzar el juego, se deberá mostrar un conjunto de categorías, por ejemplo:

- Edad Famosos
- Duración Películas
- Estreno Películas
- Dinero Famosos.
- Etc *(sean creativos!!!!)*

El jugador elegirá una categoría, según la categoría elegida debería adivinar un atributo mayor o menor de acuerdo con la pregunta.



Supongamos que se elige “Dinero Famosos”, la pregunta de esa categoría podría ser:
“¿Qué famoso es más rico?” Y presentar la opción:
“1. Luciana Aymar / 2. Lionel Messi.”



Supongamos que el jugador responde 2. Lionel Messi y que la respuesta es correcta, en ese momento el programa debe mostrar los montos.
"*Luciana Aymar 10 / Lionel Messi 20*."
Se le da a continuar y el sistema muestra ahora la opción correcta, en este caso:
2. Lionel Messi / 3. Cristiano Ronaldo.
El jugador elige una respuesta la cual puede ser correcta o no. La siguiente opción será entre Cristiano Ronaldo y otra persona, y así sucesivamente hasta completar las 6 opciones.
Si la respuesta es correcta suma 1 punto, si es incorrecta no suma.
Como idea, las categorías y las preguntas pueden ser muy variadas, puede ser categoría "Edad Famosos" y la pregunta "¿Quien nació antes?", categoría "Países" y la pregunta "¿Qué país tiene más habitantes?, como categoría "Música" como pregunta "¿Qué banda o cantante tiene más visitas en Spotify"; y muchas más?
Se harán siempre, sí o sí, 6 posibles respuestas por partida y para ganar el juego se tienen que contestar al menos 4 bien, de lo contrario se pierde.



Cada opción en cada ronda debe ser aleatoria dentro de la categoría elegida.



Aplicar las funciones de la libreria "random" de Python.

Se deberá registrar el jugador en el archivo JUGADORES.DAT con la cantidad de veces que ganó y que perdió. Si el nombre del jugador ya existe no se deberá crear otro jugador con el mismo nombre sino actualizar el registro.



B. Número secreto:

El programa debe generar un número aleatorio y no mostrarlo.
El jugador tendrá 5 intentos de adivinar el número. Cada vez que el jugador indique un número, si no adivina el programa deberá indicar si el número que pensó la máquina es mayor o menor. El jugador, deberá a ingresar otro número para intentar adivinar.
Previo a que el jugador ingrese un número, la máquina deberá mostrar cuantos intentos le queda por adivinar.
Al final, si adivina mostrar la cantidad de intentos que le llevó adivinar y, en caso de no adivinar mostrar un mensaje indicando que el jugador perdió y el número que pensó la máquina.
Se deberá llevar registro de la cantidad de veces que cada usuario jugó, la cantidad que acertó y la cantidad de veces que perdió el juego.
Aquí también usar la función de Python “random” para la generación del número a adivinar.
Se deberá registrar el jugador en el archivo JUGADORES.DAT con la cantidad de veces que ganó y que perdió. Si el nombre del jugador ya existe no se deberá crear otro jugador con el mismo nombre sino actualizar el registro.

C. Blackjack:

El Blackjack, también conocido en muchos países como “El 21”, es uno de los juegos de cartas más populares del mundo. Se juega, tradicionalmente, en los casinos con una o más barajas inglesas (cartas de Poker) de 52 cartas.
A diferencia de otros juegos de cartas donde compites contra los demás jugadores de la mesa, en el Blackjack tu único rival es la casa (representada por el crupier o la banca).

Reglas del Juego “Simplificado”

- **Valor de las Cartas:** Las cartas del 2 al 10 valen su propio número.
  - Las cartas con figuras (J, Q, K) valen **10 puntos**.
  - El **As (A)** valdrá **11 puntos** o **1 punto** según le convenga al jugador.
- **El Turno del Jugador:** El juego comienza donde el programa te reparte **dos cartas** al azar.
  - El programa deberá mostrarte tus cartas y la suma total de tus puntos.
  - El programa te preguntará: *¿Quieres "Pedir" otra carta o "Plantarte"?*
  - Si pides carta, se te suma otra carta al azar. Si te pasas de 21, ¡pierdes automáticamente!
  - Si te plantas, tu turno termina y se guarda tu puntuación actual.

- **El Turno de la Banca:**
    - Una vez que te plantas, la Banca juega automáticamente.
    - Para simular la inteligencia de la banca, esta **debe pedir cartas obligatoriamente mientras tenga menos de 17 puntos**. Si llega a 17 o más, se planta automáticamente.
- **¿Quién gana?**
    - Si te pasaste de 21, gana la Banca.
    - Si la Banca se pasa de 21 (y tú no), ganas tú.
    - Si ninguno se pasó, gana el que tenga la puntuación más alta. En caso de tener los mismos puntos, es un **Empate**.

Cuando el juego comienza se muestra una de las dos cartas de la banca (computadora) y las dos cartas del jugador. Recordar que son 52 cartas, es decir 4 palos y 13 cartas por palo, 2 al 10, J, Q, K, A. Validar que se juega con un solo mazo de cartas y que no puede haber cartas repetidas.
Una vez repartidas 2 cartas a cada uno, el jugador verá sus cartas y una de la de la banca, el usuario puede decidir si pedir o no. Preguntar al usuario que quiere hacer. Si el usuario pide carta (opción "Pedir") esta se suma a las anteriores, si la suma es menor a 21 puede volver a pedir, es mayor pierde automáticamente y si es igual pasa el turno a la banca.
Si la opción es "Plantarse" o si la suma da 21, se reparten automáticamente cartas a la banca. La banca juega con las siguientes reglas, si la suma de sus cartas es 16 o menos, pide carta, si la suma es 17 o más se planta.
Finalizada la partida, el programa debe consultar si quiere jugar otra partida o no. Si quiere, comienza automáticamente una nueva partida, caso contrario, vuelve al menú principal.
Se tiene que llevar la cantidad de veces que gana el jugador.
Se deberá registrar el jugador en el archivo JUGADORES.DAT con la cantidad de veces que ganó y que perdió. Si el nombre del jugador ya existe no se deberá crear otro jugador con el mismo nombre sino actualizar el registro.

D. Par o impar:

También en este caso el programa debe solicitar al jugador, su nombre. Verificar su existencia, de lo contario dar de alta a un nuevo jugador.
En esta primera etapa el programa deberá generar 2 números aleatorios del 1 al 6 y sumarlos.
Sin mostrarlos el usuario debe indicar, a solicitud del programa, si la suma de los números es par o impar. Si acierta, mostrar que ganó y si pierde mostrar que perdió.
Ahora si implementaremos el crédito para el usuario. Cada usuario comenzará con un crédito de $ 10000 y deberá apostar una suma no mayor a lo que tiene, si acierta se le sumará de crédito tanto como apostó, si no acierta, se le resta. El usuario no puede apostar más de lo que tiene y si se queda con cero ya no podrá jugar.
Se debe llevar la cuenta de la cantidad de veces que acertó.
Se deberá registrar el jugador en el archivo JUGADORES.DAT con la cantidad de veces que ganó y que perdió. Si el nombre del jugador ya existe no se deberá crear otro jugador con el mismo nombre sino actualizar el registro.

E. Reporte:

Si se elige esta opción el programa deberá mostrar un submenú iterativo:

a. Lista de jugadores ordenados de mayor a menor por cantidad de créditos que tiene.
b. Juegos jugados por un jugador. En este caso el programa solicita un nombre de jugador. El programa debe informar a que juegos jugó, cuantas veces ganó, cuantas veces perdió a cada juego y cuantos créditos le quedan.
c. Volver al menú principal

F. Administración de Juegos:

Se deberá mostrar el siguiente submenú.

1. Administrar Categorías.
2. Administrar Opciones.
3. Volver: Vuelve al menú anterior

Y dentro Administrar Categorías:

1- Alta
2- Modificación
3- Baja
4- Volver: Vuelve al menú anterior.

En Administrar Opciones:

1- Alta
2- Consulta
3- Volver

Cuando el usuario ingrese la opción F, el sistema le pedirá una contraseña, es una contraseña de administrador para poder gestionar categorías y opciones para el juego del mayor/menor. En ese caso se debe comparar simplemente contra una *constante* del programa principal. Al momento en el que el usuario ingresa la contraseña la misma tiene que ser *oculta*, por ejemplo, mostrando asteriscos o simplemente no mostrar que se está tipeando. Para esto, pueden utilizar alguna librería que lo permita. La cantidad de veces que puede ingresar la contraseña son 3. Sino, debe retornar al menú con un mensaje *"Superó los 3 intentos de ingresar contraseña, salga e intente nuevamente"*

Si la contraseña es correcta se accede al menú;

- **Categoría – Alta:** Se debe ingresar un nombre de categoría. El sistema debe validar que no haya otra categoría con el mismo nombre (si se escribe con mayúsculas y/o minúsculas es el mismo nombre). Si pasa la validación debe ingresar la pregunta, por ejemplo: ¿Quién tiene más años?

Finalmente, el sistema debe generar un número de categoría consecutivo secuencial al partir del último y registrar la nueva categoría y su pregunta y en estado “A” Activa.

- **Categoría – Modificación**: El sistema debe mostrar un listado de todas las categorías que estén en estado Activa “A”, número y nombre. Luego el usuario ingresa un número y el sistema debe permitir la modificación del nombre. Una vez modificado debe registrar el nuevo nombre.
- **Categoría – Baja**: El sistema debe mostrar un listado de todas las categorías que estén en estado Activa “A”, número y nombre. Luego, el usuario ingresa un número y el sistema debe cambiar el estado a Inactiva “I”.

Nota: tanto para la modificación como para la baja se debe validar que el estado del número ingresado sea “A”.

- **Opciones – Alta**: El programa debe mostrar un listado con todas las categorías Activas “A”. En el caso que aún no haya categorías, debe mostrar un cartel informando que antes de registrar opciones se deben dar de alta categorías.

Luego, el usuario, debe ingresar el número de la categoría y el programa solicitará las opciones a la pregunta y respuesta numérica.

Por ejemplo: Se selecciona la categoría 3 – Edades y el programa solicita, por ejemplo: Objeto: *Tom Cruise*, Valor: *61*.

El programa deberá guardar un registro nuevo en Opciones. Por ejemplo, con los valores: 3 es la categoría elegida, 1 es el número de Opción, Objeto es Tom Cruise y 61 es el valor. Recordar, que el número de categoría elegida es lo que relaciona un registro del archivo de Opciones con un registro del archivo de Categorías.

- **Opciones - Consulta**: El sistema debe mostrar un listado de categorías, número y nombre. Luego el usuario ingresa un número de categoría y el sistema muestra la pregunta, objetos de la pregunta y respuestas numéricas.

G. Salir del programa:

Mostrar un cartel que diga: *"Gracias por jugar, no apueste, juega por diversión"* y luego de presionar "enter", el programa se cierra.

Notas:

Importante!! Respetar los principios de la programación estructurada modular descendente.

Utilizar archivos de acceso directo, ni de texto, ni indexado utilizando las librerías pickle como se indica en la bibliografía de la cátedra y se desarrolla en clases.

No se admitirán sentencias como: **break, exit(), while true, return si no es una función que devuelve un valor al final de la misma,** …

- En Python:
  - NO usar librerías externas no permitidas
  - NO usar listas ni diccionarios ni ninguna otra estructura de datos que no se haya dado en el curso.
  - NO usar sort, find, join o ningún método similar.
  - El programa debe compilar correctamente.
- TODOS los integrantes del equipo deben entregar EL MISMO trabajo.
- TODOS los archivos deben estar en la carpeta c:\\tp3\\
- En caso de entregar un link a Google Drive, la última fecha de modificación debe ser anterior a la fecha límite de entrega.
- NO se aceptan entregas por e-mail o por WhatsApp ni ningún otro medio que no sea CVG.